Does Low-Communication Training Survive Bad Networks? A Reproducible Study of DDP, DiLoCo and Hierarchical Aggregation over Emulated WAN Links

Divij Karanam
Independent Researcher

***Abstract*—Training neural networks across geographically distributed sites is limited by slow, lossy wide-area links. Low-communication methods such as DiLoCo synchronize rarely, and hierarchical schemes aggregate within regions before crossing the WAN. We present a reproducible, CPU-scale study that trains a small transformer language model with four workers in two emulated regions and compares synchronous data parallelism (DDP), DiLoCo, and a two-level hierarchical DiLoCo, each with FP32, INT8 or INT4 compression of everything that crosses the WAN. Synchronization rounds are measured over real TCP connections through a packet-level emulator with per-region full-duplex links, finite buffers and a 1500-byte MTU, under four WAN profiles from 100 Mbit/s to 2 Mbit/s with 1% loss. With the same 40 WAN synchronizations, hierarchical DiLoCo reached a validation loss of 1.665 versus 1.696 for flat DiLoCo, and it also beat flat DiLoCo synchronizing five times as often (1.689). INT8 and INT4 compression changed final loss by at most 0.038. DDP reached the best loss (1.587) but, on a 10 Mbit/s link, would spend an estimated 186 minutes on 2,000 steps versus 5.4 minutes for hierarchical DiLoCo with INT8. A real four-process distributed run matched the simulation's validation loss to within 0.031, and the wall-clock model underestimated its total run time by 8–15%. Code and data are released.**

***Index Terms*—distributed training, low-communication training, DiLoCo, local SGD, hierarchical aggregation, gradient compression, network emulation**

I. Introduction

Data-parallel training synchronizes gradients after every step, which assumes datacenter interconnects. When compute is spread across regions or organizations connected by the public internet, this assumption fails: a single synchronization of a model's parameters can take seconds over a congested or lossy wide-area network (WAN). Low-communication methods address this. DiLoCo [1] lets each worker take hundreds of local optimizer steps and then averages "pseudo-gradients" with an outer Nesterov-momentum step, and it has been used to train models across continents [2], [3]. Hierarchical variants first aggregate inside a region over fast local links and send only one update per region over the WAN [4], [5].

Most evaluations of these methods run on large GPU clusters and model WAN effects analytically. This paper asks a narrower, reproducible question: *under realistic packet-level WAN behavior, how do synchronous, low-communication and hierarchical training compare in time and model quality?* Our contributions are:

1) An open testbed that runs on a single 4-core machine: a small transformer trained by four workers in two regions, real TCP synchronization rounds, and a WAN emulator with per-region full-duplex links, drop-tail buffers and a realistic MTU.
2) A comparison of DDP, DiLoCo and two-level hierarchical DiLoCo with FP32, INT8 and INT4 compression across four WAN profiles, with tuned hyperparameters and two seeds per configuration.
3) The finding that, at equal WAN synchronization counts, hierarchical aggregation improves model quality, while 8- and 4-bit compression is nearly free.
4) An end-to-end validation with real distributed processes, and a description of emulation pitfalls (TCP Small Queues, loopback MTU) that can distort such studies, together with a check for TCP slow start after idle.

II. Related Work

Local SGD and federated averaging [6], [7] reduce communication by averaging models after several local steps. Lin et al. [5] proposed hierarchical local SGD, which synchronizes frequently within groups and rarely across them. DiLoCo [1] combines many inner AdamW [8] steps with an outer Nesterov step [9]. Streaming DiLoCo [10] synchronizes parameter fragments and overlaps communication with computation, and OpenDiLoCo [2] and INTELLECT-1 [3] demonstrated globally distributed training, the latter with INT8 pseudo-gradients. DeMo [11] reduces communication by synchronizing only fast-moving momentum components. HALoS [4] introduces hierarchical asynchronous local SGD for geo-distributed LLM training. Gradient compression has a long history, including 1-bit SGD [12], QSGD [13] and deep gradient compression [14]. Our study differs in scale and emphasis: it is small enough to reproduce on a laptop-class CPU, and it measures synchronization over real TCP connections through a packet-level emulator.

III. Method

A. Model, data and training

The model is a GPT-style character-level transformer (3 layers, width 96, 4 heads, context 64; 351,744 parameters) trained on Tiny Shakespeare [15] (90/10 train/validation split). Four workers each sample batches of 16 sequences from a disjoint contiguous shard of the training text, so their data distributions differ mildly. Every method runs 2,000 steps per worker with AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0, 100 warm-up steps, cosine decay to 10%). Validation loss is computed on a fixed set of 512 held-out sequences. Training is implemented in JAX [16]; workers are simulated with vectorization, which is mathematically identical to separate machines.

B. Synchronization strategies

DDP averages gradients across the four workers after every step (2,000 synchronizations). **DiLoCo** with period H: every H steps, each worker i forms the pseudo-gradient $\Delta_i = \theta_g - \theta_i$, the average $\bar{\Delta}$ is applied to the global parameters θ_g with outer SGD with Nesterov momentum (momentum 0.9), and every worker restarts from θ_g. Inner optimizer states remain local. **Hierarchical DiLoCo** groups the workers into two regions of two. Every H_r steps, workers average their parameters within their region over the local link. Every H_g steps, each region sends one pseudo-gradient over the WAN to a global DiLoCo outer step. We use $H_r = 10$ and $H_g \in \{50, 100\}$, compared with flat DiLoCo at $H \in \{10, 50, 100\}$.

C. Compression

Everything that crosses the WAN, both the uploaded pseudo-gradients and the averaged result sent back, is quantized blockwise (256 values per block, one FP32 scale each) to INT8 or INT4 with symmetric round-to-nearest. Payloads are 1407 kB (FP32), 357 kB (INT8) and 181 kB (INT4). Local-link traffic in the hierarchical scheme is FP32.

D. Hyperparameter tuning

On seed 0 we swept the DDP learning rate over $\{1, 2, 4, 8, 16\} \times 10^{-3}$, and the DiLoCo ($H$=50) inner and outer learning rates. The sweep was extended twice because the best values initially lay at the edge of the range. The final choices were 1.6×10^{-2} for DDP, and 1.6×10^{-2} (inner) with 0.7 (outer) for DiLoCo; DDP's loss differed by only 0.0002 between 8×10^{-3} and 1.6×10^{-2}. Hierarchical DiLoCo reuses DiLoCo's values without further tuning. Reported results use seeds 1 and 2.

E. Network testbed

Each region reaches a central coordinator over its own emulated WAN link, while workers reach their regional hub over an unimpaired local link. A userspace emulator built on Linux NFQUEUE gives each link two independent directions. Each direction serializes packets at the link rate, adds propagation delay, drops packets at random with the configured probability, and enforces a drop-tail buffer of one bandwidth-delay product. The loopback MTU is set to 1500 bytes during measurements, and TCP uses Linux defaults (CUBIC [17]). Table I lists the WAN profiles. A synchronization round is timed from the moment workers start sending to the moment all workers hold the averaged result. Rounds are separated by 1 s, so TCP slow start after idle applies, as it would between DiLoCo synchronizations. Because DDP synchronizes after every step, its rounds were also measured back to back (50 ms gaps), so that TCP congestion windows stay open; these rounds were within 6% of rounds with idle gaps.

TABLE I
WAN profiles (per direction of each region's link).

Profile	Rate	One-way delay	Loss
Regional	100 Mbit/s	10 ms	0%
Intercontinental	50 Mbit/s	75 ms	0.1%
Constrained	10 Mbit/s	100 ms	0.5%
Poor	2 Mbit/s	150 ms	1%

F. Wall-clock model and validation

Because every strategy is synchronous, its convergence per step does not depend on the network. We therefore estimate training time as

$$T = S\,t_c + N_{\mathrm{WAN}}\,r_{\mathrm{WAN}} + N_{\mathrm{LAN}}\,r_{\mathrm{LAN}}, \qquad (1)$$

where $S = 2{,}000$ steps, $t_c = 141$ ms is the measured per-worker compute time per step (four processes, one core each), N is the number of synchronizations of each kind, and r is the measured median round time. To validate this decomposition, we ran DiLoCo (H=50, 300 steps) as four real worker processes and a coordinator exchanging real quantized payloads over the constrained profile, and compared the result with the simulation.

IV. Results

A. Model quality

Table II and Fig. 1 summarize final validation loss. DDP reached 1.587. With the same 40 WAN synchronizations, hierarchical DiLoCo (10/50) reached 1.665, better than flat DiLoCo (H=50, 1.696), and also better than flat DiLoCo synchronizing five times as often (H=10, 1.689). The same ordering held at 20 WAN synchronizations (hierarchical 10/100: 1.779; DiLoCo H=100: 1.826). A likely explanation is that frequent, cheap intra-region averaging reduces the divergence between workers, so each regional pseudo-gradient is a better estimate of the shared update.

B. Compression

INT8 and INT4 quantization of WAN traffic changed final loss by at most 0.038 across the DiLoCo and hierarchical configurations, a difference comparable to the variation between seeds (Fig. 2). Compression therefore cuts WAN bytes by 4–8× at little or no measurable cost at this scale.

TABLE II
FINAL VALIDATION LOSS (MEAN OF SEEDS 1 AND 2; RANGE IN BRACKETS) AND NUMBER OF WAN SYNCHRONIZATIONS.

Configuration	WAN syncs	Loss	[min, max]
DDP (every step)	2,000	1.587	[1.582, 1.593]
DiLoCo H=10	200	1.689	[1.679, 1.698]
DiLoCo H=50	40	1.696	[1.694, 1.698]
DiLoCo H=50, INT8	40	1.703	[1.691, 1.716]
DiLoCo H=50, INT4	40	1.703	[1.687, 1.718]
Hierarchical 10/50	40	1.665	[1.661, 1.670]
Hierarchical 10/50, INT8	40	1.661	[1.656, 1.666]
Hierarchical 10/50, INT4	40	1.682	[1.681, 1.684]
DiLoCo H=100	20	1.826	[1.808, 1.845]
DiLoCo H=100, INT8	20	1.865	[1.839, 1.890]
DiLoCo H=100, INT4	20	1.841	[1.822, 1.861]
Hierarchical 10/100	20	1.779	[1.770, 1.789]
Hierarchical 10/100, INT8	20	1.786	[1.786, 1.787]
Hierarchical 10/100, INT4	20	1.791	[1.791, 1.792]

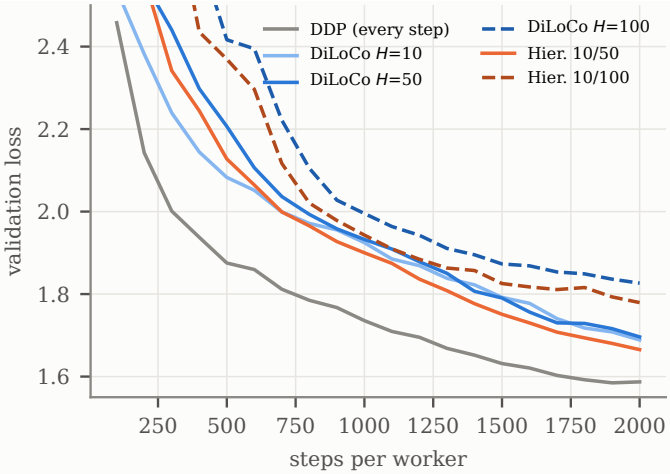


Fig. 1. Validation loss during training (FP32, mean of two seeds).

C. Synchronization cost

Table III lists measured round times. Hierarchical aggregation roughly halves FP32 round time on the bandwidth-limited profiles (regional, constrained, poor), because each region link carries one update instead of two; on the latency-bound intercontinental profile the reduction is 28%. Compression helps most on low-bandwidth links. On high-latency links, rounds are dominated by round trips and TCP slow start rather than bytes: on the intercontinental profile, INT8 payloads are 4× smaller than FP32 but only about 2.0× faster.

D. Time versus quality

Fig. 3 combines quality and estimated training time. Computation alone takes 4.7 minutes for 2,000 steps, so low-communication methods are close to compute-bound on all but the poor profile. DDP is the slowest on every profile, by a factor that grows as the network worsens: 4.5× slower than hierarchical 10/50 with INT8 on the regional profile, 9.5× on the intercontinental, 34× on the constrained (186 versus 5.4 minutes) and 128× on the poor profile (about 15 hours versus

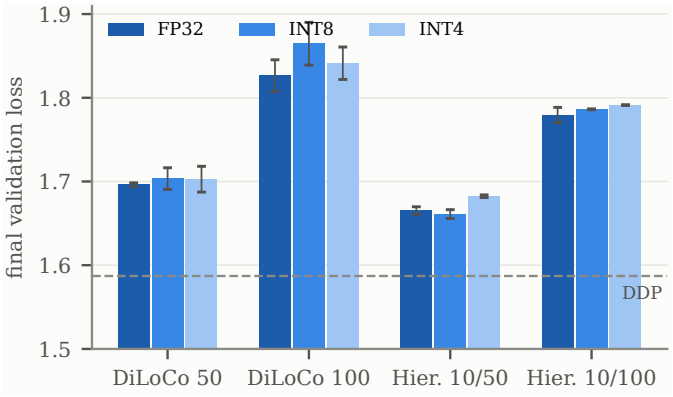


Fig. 2. Effect of WAN compression on final validation loss (bars: mean of two seeds; whiskers: range).

TABLE III
MEDIAN SYNCHRONIZATION ROUND TIME IN SECONDS (MEASURED).

	DDP	Flat			Hierarchical		
Profile	FP32	FP32	INT8	INT4	FP32	INT8	INT4
Regional	0.51	0.51	0.19	0.08	0.27	0.09	0.06
Intercont.	1.32	1.40	0.69	0.42	1.01	0.57	0.44
Constrained	5.45	5.46	1.83	1.19	2.79	1.06	0.98
Poor	27.07	26.50	6.86	4.14	12.84	3.54	3.05

7.1 minutes). Hierarchical 10/50 with INT8 has the second-best quality after DDP and lies on the time–quality frontier on all four profiles. DDP is preferable only when its quality advantage justifies its network cost.

E. Validation

The real four-process DiLoCo runs reached validation losses of 2.534 (FP32) and 2.517 (INT8), versus 2.541 and 2.548 in simulation (difference at most 0.031). Their measured synchronization rounds had medians of 5.99 s (FP32) and 2.50 s (INT8), longer than the benchmark's 5.46 s and 1.83 s, because in real training workers enter a round at slightly different times and payloads must be encoded and decoded. As a result, the wall-clock model underestimated the real runs' total time by 8% (FP32) and 15% (INT8). These errors are small relative to the differences between methods, but the estimates in Fig. 3 should be read as lower bounds.

V. DISCUSSION AND LIMITATIONS

Several emulation details mattered. Queueing delayed packets on the sending side of loopback triggered TCP Small Queues and capped throughput at about 4 Mbit/s on a 100 Mbit/s link, so the emulator queues on the receive path. With loopback's default 64 kB MTU, TCP would react far less to packet loss than on real paths [18]. We also measured DDP rounds back to back to rule out a penalty from TCP slow start after idle [19]; in our setting the effect was negligible.

The study is deliberately small: a 352K-parameter character-level model, 2,000 steps, four workers and two regions. The ratio of computation to communication differs from LLM

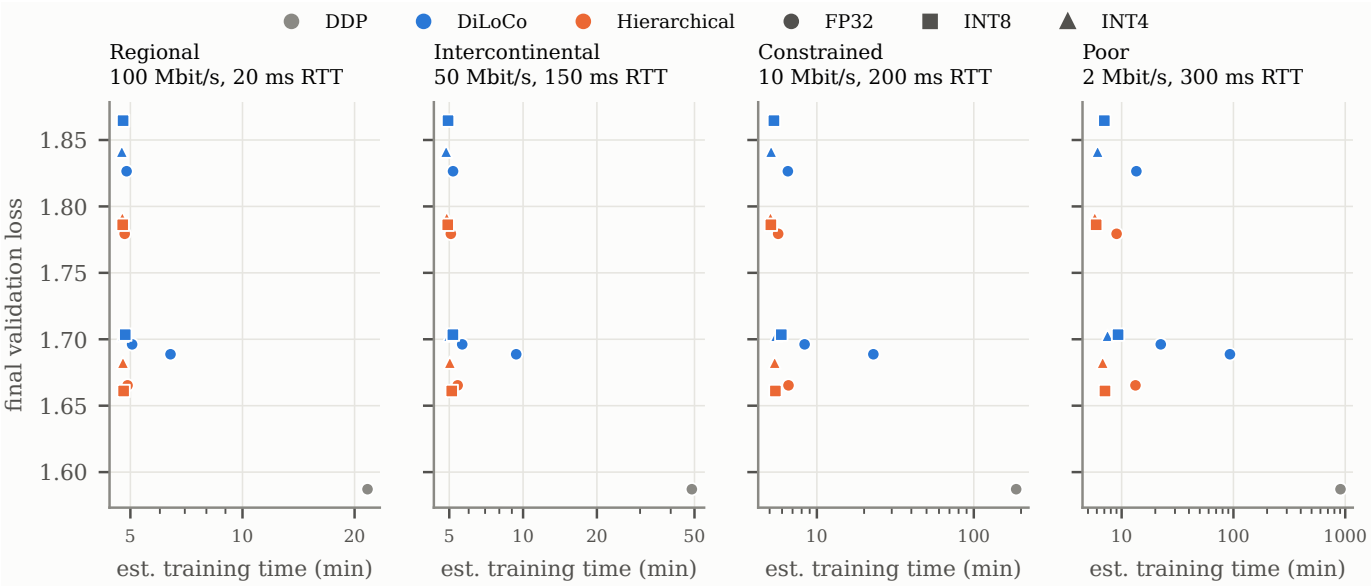


Fig. 3. Estimated time for 2,000 steps (log scale) versus final validation loss for each configuration and WAN profile. Lower-left is better.

training, where models are larger and steps slower, and DiLoCo has been reported to match synchronous training at larger scale [1]. Hyperparameters were tuned for DDP and flat DiLoCo, and the hierarchical scheme reused DiLoCo's values, so its advantage is not due to extra tuning. Local links were unimpaired, the WAN was emulated rather than real, and results are from two seeds.

VI. Conclusion

Under realistic WAN behavior, synchronous data parallelism is dominated by communication. Hierarchical DiLoCo with INT8 compression reduced network time by 109–379× relative to DDP across the four profiles, at a moderate quality cost. Hierarchical aggregation narrows that cost at no extra WAN traffic, and 8- or 4-bit compression is nearly free. Combining rare WAN synchronization, regional aggregation and compression is a practical recipe for training over bad networks.

Data Availability

Code, raw results and the network emulator are available in the Zenodo record of this preprint.

Acknowledgment

The author used Claude (Anthropic), an AI assistant, to help write the code, run the experiments and draft this manuscript. The author is responsible for the content of the paper.

References

[1] A. Douillard *et al.*, "DiLoCo: Distributed low-communication training of language models," arXiv:2311.08105, 2023.
[2] S. Jaghouar, J. M. Ong, and J. Hagemann, "OpenDiLoCo: An open-source framework for globally distributed low-communication training," arXiv:2407.07852, 2024.
[3] S. Jaghouar *et al.*, "INTELLECT-1 technical report," arXiv:2412.01152, 2024.
[4] G.-W. Kim *et al.*, "HALoS: Hierarchical asynchronous local SGD over slow networks for geo-distributed large language model training," arXiv:2506.04531, 2025.
[5] T. Lin, S. U. Stich, K. K. Patel, and M. Jaggi, "Don't use large mini-batches, use local SGD," in *Proc. ICLR*, 2020.
[6] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.
[7] S. U. Stich, "Local SGD converges fast and communicates little," in *Proc. ICLR*, 2019.
[8] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.
[9] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *Proc. ICML*, 2013.
[10] A. Douillard *et al.*, "Streaming DiLoCo with overlapping communication: Towards a distributed free lunch," arXiv:2501.18512, 2025.
[11] B. Peng, J. Quesnelle, and D. P. Kingma, "DeMo: Decoupled momentum optimization," arXiv:2411.19870, 2024.
[12] F. Seide, H. Fu, J. Droppo, G. Li, and D. Yu, "1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs," in *Proc. Interspeech*, 2014.
[13] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
[14] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *Proc. ICLR*, 2018.
[15] A. Karpathy, "char-rnn (tiny shakespeare dataset)," https://github.com/karpathy/char-rnn, 2015.
[16] J. Bradbury *et al.*, "JAX: composable transformations of Python+NumPy programs," https://github.com/jax-ml/jax, 2018.
[17] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, 2008.
[18] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 27, no. 3, 1997.
[19] J. Chu, N. Dukkipati, Y. Cheng, and M. Mathis, "Increasing TCP's initial window," RFC 6928, 2013.